

Сравнение классов с `slots` и `weakref` в Python

Результаты эксперимента

Были реализованы три варианта класса `Knight` с одинаковыми полями `hp`, `damage`, `speed`:

- **KnightPlain** — обычный класс (атрибуты в `__dict__`);
- **KnightSlots** — класс с `__slots__`;
- **KnightSlotsWeak** — класс с `__slots__` и поддержкой `weakref`.

Для каждого варианта было создано по 1 000 000 объектов, измерены время создания, время чтения/изменения атрибутов и пиковое использование памяти. Для варианта с `weakref` отдельно замерено время создания слабых ссылок.

Сводные численные результаты представлены в табл. 1.

Таблица 1: Сравнение времени и памяти для трёх реализаций класса `Knight` ($n = 1\,000\,000$)

Класс	Время создания, с	Время чтения/изменения, с	Время <code>weakref</code> , с	Пиковая память, MiB
KnightPlain	0.4315	0.1175	—	130.12
KnightSlots	0.5113	0.1154	—	91.98
KnightSlotsWeak	0.5009	0.1153	0.2416	107.24

Более подробные профили по времени (`cProfile`) и памяти (`tracemalloc`) приведены на приложенных скриншотах консольного вывода.

Выводы

По таблице и профилировщику можно сделать следующие наблюдения:

- Время создания немного меньше у обычного класса, в то время как чтение/изменение атрибутов у всех трёх реализаций очень близки; небольшие отличия лежат в пределах погрешности.
- Класс **KnightSlots** показал наименьшее пиковое использование памяти (**91.98 MiB**) по сравнению с **KnightPlain** (130.12 MiB) и **KnightSlotsWeak** (107.24 MiB). Это соответствует теории: `__slots__` уменьшают накладные расходы на объект за счёт отсутствия словаря `__dict__`.
- Вариант **KnightSlotsWeak** по памяти занимает второе место: добавление `__weakref__` слегка увеличивает размер объекта по сравнению с чистыми слотами, но остаётся заметно экономичнее обычного класса.
- Для варианта **KnightSlotsWeak** отдельно измерялась стоимость создания слабых ссылок: создание 1 000 000 объектов `weakref` на основе уже существующих рыцарей заняло около 0.24 с. занимает около **0.24 с.**

В целом результаты согласуются с теоретическими ожиданиями: `__slots__` дают выигрыш по памяти при большом количестве однотипных объектов, при этом разница по времени доступа к атрибутам между реализациями в реальном коде получается небольшой. Поддержка `weakref` немного увеличивает накладные расходы, но позволяет использовать слабые ссылки, когда это необходимо.

```

KnightPlain (n=1000000)
Создание объектов: 0.431522 с
Чтение/изменение: 0.117499

KnightSlots (n=1000000)
Создание объектов: 0.511306 с
Чтение/изменение: 0.115388

KnightSlotsWeak (n=1000000)
Создание объектов: 0.500858 с
Чтение/изменение: 0.115269
Слабые ссылки: 0.241629 с (объектов: 1000000)

```

Рис. 1: Результаты бенчмарка по времени

```

=== Профилирование вызовов: KnightPlain (n=1000000) ===
2000003 function calls in 0.797 seconds

Ordered by: internal time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1   0.570    0.570    0.688    0.688 /Users/leonidlevin/Documents/VK_Python_study/VK_Python/08/knight_benchmark.py:45(spawn_knights)
      1   0.110    0.110    0.110    0.110 /Users/leonidlevin/Documents/VK_Python_study/VK_Python/08/knight_benchmark.py:59(work_with_knights)
1000000   0.069    0.000    0.069    0.000 {method 'append' of 'list' objects}
1000000   0.049    0.000    0.049    0.000 /Users/leonidlevin/Documents/VK_Python_study/VK_Python/08/knight_benchmark.py:11(__init__)
      1   0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}

=== Профилирование вызовов: KnightSlots (n=1000000) ===
2000003 function calls in 0.775 seconds

Ordered by: internal time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1   0.550    0.550    0.667    0.667 /Users/leonidlevin/Documents/VK_Python_study/VK_Python/08/knight_benchmark.py:45(spawn_knights)
      1   0.108    0.108    0.108    0.108 /Users/leonidlevin/Documents/VK_Python_study/VK_Python/08/knight_benchmark.py:59(work_with_knights)
1000000   0.069    0.000    0.069    0.000 {method 'append' of 'list' objects}
1000000   0.048    0.000    0.048    0.000 /Users/leonidlevin/Documents/VK_Python_study/VK_Python/08/knight_benchmark.py:21(__init__)
      1   0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}

=== Профилирование вызовов: KnightSlotsWeak (n=1000000) ===
2000003 function calls in 0.794 seconds

Ordered by: internal time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1   0.566    0.566    0.684    0.684 /Users/leonidlevin/Documents/VK_Python_study/VK_Python/08/knight_benchmark.py:45(spawn_knights)
      1   0.110    0.110    0.110    0.110 /Users/leonidlevin/Documents/VK_Python_study/VK_Python/08/knight_benchmark.py:59(work_with_knights)
1000000   0.069    0.000    0.069    0.000 {method 'append' of 'list' objects}
1000000   0.049    0.000    0.049    0.000 /Users/leonidlevin/Documents/VK_Python_study/VK_Python/08/knight_benchmark.py:31(__init__)
      1   0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}

```

Рис. 2: Результаты профилирования вызовов (cProfile)

```
=== Память: KnightPlain (n=1000000) ===  
Пиковая память: 130.12 MiB  
  
=== Память: KnightSlots (n=1000000) ===  
Пиковая память: 91.98 MiB  
  
=== Память: KnightSlotsWeak (n=1000000) ===  
Пиковая память: 107.24 MiB
```

Рис. 3: Результаты профилирования памяти (tracemalloc)